

NATIONAL ECONOMICS UNIVERSITY, HANOI
COLLEGE OF TECHNOLOGY
FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE



DATABASE SYSTEMS

REPORT

Topic: Personal Finance Management System

Instructor: Tran Hung

Student: Dinh Quoc Viet 11247150

Hanoi, 2026

Contents

1	Introduction	2
1.1	Project Overview	2
1.2	Project Objectives	2
1.3	System Requirements	2
1.4	Scope of the Project	2
1.5	Tools and Technologies	3
2	Database Design	3
2.1	Entity-Relationship (ER) Modeling	3
2.2	Relational Schema	3
2.3	Table Structures & Constraints	4
2.4	Relationship Design	4
3	Database Implementation	5
3.1	Sample Data Injection	5
3.2	Indexing Strategy	5
3.3	SQL Views	6
3.4	Stored Procedures	6
3.5	User-Defined Functions	7
3.6	Automated Triggers	7
3.7	Backup & Recovery Protocols	7
4	Python Application Development	7
4.1	Database Connection Configuration	7
4.2	Main CLI Application	7
4.3	Validation and Data Integrity Mechanisms	8
4.4	Financial Analytics and Visualization	8
5	Database Security and Administration	10
5.1	Comprehensive Audit Trails	10
5.2	Access Control and Privileges	10
5.3	Performance Optimization	10
6	Conclusion and Recommendations	10
6.1	Conclusion	10
6.2	Recommendations for Future Work	10
7	References	11

1 Introduction

1.1 Project Overview

The Personal Finance Management System is a database-driven application engineered to assist individuals in systematically organizing and monitoring their financial activities. The system delivers a seamless workflow for tracking incomes, categorizing daily expenditures, managing bank accounts, and analyzing financial health through automated summaries. To ensure robust data tracking, the system integrates an automated balance history log to record every account fluctuation.

1.2 Project Objectives

The primary goal of this project is to architect a reliable system that empowers users to oversee their personal finances effectively. Specifically, it aims to track spending habits, evaluate budget limits, and optimize savings through data-driven reporting. This solution is built utilizing MySQL for robust database management and Python for an interactive Command-Line Interface (CLI) application.

1.3 System Requirements

The developed system successfully fulfills the following core requirements:

- Secure storage and management of users, income streams, expenses, bank accounts, and spending categories.
- Interactive CLI for data entry, including categorized expenses and income logging.
- Continuous monitoring of bank accounts with a detailed, trigger-driven balance history.
- Generation of monthly financial summaries and real-time budget status checks.
- Visual reporting capabilities (Bar and Pie charts) for intuitive financial analysis.

1.4 Scope of the Project

This project emphasizes advanced relational database design, SQL implementation, and Python application development. The implemented architecture includes normalized schemas, indexing, complex views, scalar functions, stored procedures, and triggers. While web-based or mobile interfaces are considered for future expansion, the current scope focuses on delivering a high-performance, feature-rich command-line application.

1.5 Tools and Technologies

- **Database Management System:** MySQL 8.x
- **Database Administration Tool:** MySQL Workbench
- **Programming Language:** Python 3.x
- **Python Libraries:** `mysql-connector-python` (Database Connection), `matplotlib` (Data Visualization)

2 Database Design

2.1 Entity-Relationship (ER) Modeling

The conceptual design of the database revolves around six core entities: **Users**, **ExpenseCategories**, **BankAccounts**, **Income**, **Expenses**, and **BalanceHistory**. A single user can hold multiple bank accounts and record numerous transactions (both income and expenses). Each expense is mapped to a specific category. Furthermore, each bank account is tied to a historical ledger (**BalanceHistory**) that logs every balance adjustment.

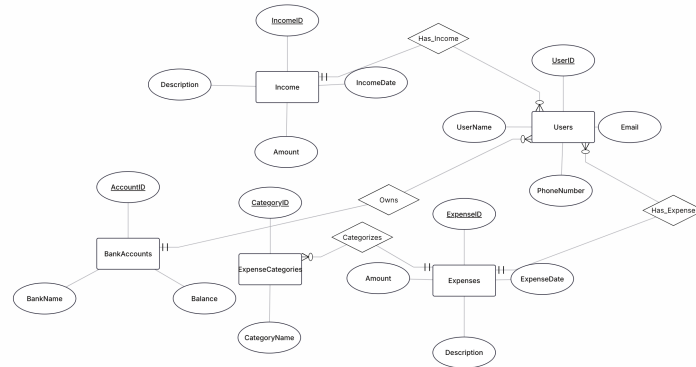


Figure 1: Entity-Relationship Diagram of the System

2.2 Relational Schema

The conceptual model translates into the following relational schemas:

- **Users** (**UserID**, **UserName**, **Email**, **PhoneNumber**)
- **ExpenseCategories** (**CategoryID**, **CategoryName**)
- **BankAccounts** (**AccountID**, **UserID**, **BankName**, **Balance**)
- **Income** (**IncomeID**, **UserID**, **AccountID**, **Amount**, **IncomeDate**, **Description**)

- **Expenses** (ExpenseID, UserID, CategoryID, AccountID, Amount, ExpenseDate, Description)
- **BalanceHistory** (HistoryID, AccountID, ChangeType, AmountChanged, BalanceBefore, BalanceAfter, ChangedAt, ReferenceType, ReferenceID, Description)

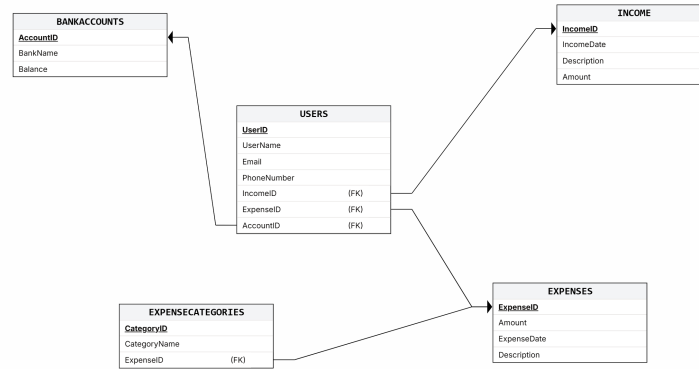


Figure 2: Relational Schema of the Database

2.3 Table Structures & Constraints

To guarantee data integrity, tables are strictly governed by Primary Keys (PK) and Foreign Keys (FK) with ON DELETE CASCADE rules. Constraints are applied to prevent negative transaction amounts, and unique constraints are enforced on user emails to avoid duplicate registrations.

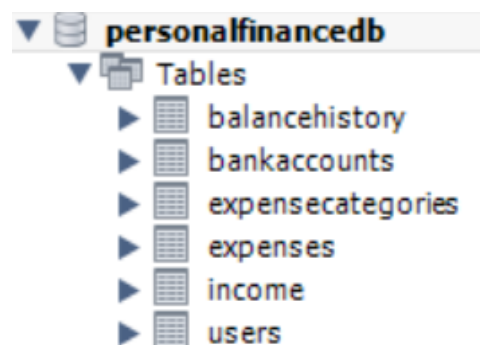


Figure 3: Main tables and constraints created in the database

2.4 Relationship Design

The relationship architecture mimics real-world financial workflows. Every financial transaction strictly requires an association with an existing bank account. To provide full auditability, the **BalanceHistory** table establishes a 1-to-N relationship with

BankAccounts, capturing the exact timestamp, transaction type, and delta of any balance modification.

3 Database Implementation

3.1 Sample Data Injection

To validate system functionalities and facilitate report generation, mock data encompassing various users, bank accounts, categories, and transactions were populated into the database.

	AccountID	UserID	BankName	Balance
▶	1	1	Vietcombank	12500000.00
	2	2	Techcombank	8400000.00
	3	3	MB Bank	25000000.00
	4	4	BIDV	5600000.00
	5	5	TPBank	18900000.00
	6	6	VietinBank	9200000.00
✱	NULL	NULL	NULL	NULL

Figure 4: Sample data injected into the BankAccounts table

	ExpenseID	UserID	CategoryID	AccountID	Amount	ExpenseDate	Description
▶	1	1	1	1	500000.00	2026-04-05	Dinner with friends
	2	1	2	1	300000.00	2026-04-10	Gasoline
	3	1	3	1	2500000.00	2026-04-15	English Course
	4	1	4	1	400000.00	2026-04-20	Cinema tickets
	5	2	1	2	600000.00	2026-04-06	Groceries
	6	3	3	3	2000000.00	2026-04-12	Books
	7	4	4	4	150000.00	2026-04-16	Netflix
	8	5	5	5	1000000.00	2026-04-20	Electricity bill
	9	6	1	6	450000.00	2026-04-22	Cafe
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 5: Sample data injected into the Expenses table

3.2 Indexing Strategy

To optimize query response times, particularly for aggregating financial reports, non-clustered indexes were established on high-frequency search columns, including UserID, AccountID, CategoryID, and transaction dates (IncomeDate, ExpenseDate).

Comment	Index_comment	Visible	Expression
		YES	NULL
		YES	NULL
		YES	NULL
		YES	NULL

Figure 6: Index definitions to optimize query performance

3.3 SQL Views

Three primary views were constructed to abstract complex JOIN operations and aggregations for the Python backend:

- **MonthlySummaryView:** Aggregates total income, total expense, and calculates net amounts on a monthly basis.
- **CategorySpendingView:** Groups and sums up expenditures by their respective categories.
- **BalanceHistoryView:** Formats and displays chronological account balance alterations.

3.4 Stored Procedures

To standardize and secure data entry, business logic for insertions was encapsulated into Stored Procedures such as **AddIncomeProc** and **AddExpenseProc**. These procedures process input parameters and ensure that transactions are safely committed to the database.

	IncomeID	UserID	AccountID	Amount	IncomeDate	Description
▶	7	1	1	800000.00	2026-04-25	Test Procedure
	6	6	6	10000000.00	2026-04-05	Salary April
	5	5	5	20000000.00	2026-04-02	Salary April
	4	4	4	8000000.00	2026-04-15	Freelance
	3	3	3	5000000.00	2026-04-10	Project Bonus
	2	2	2	12000000.00	2026-04-05	Salary April
	1	1	1	15000000.00	2026-04-01	Salary April
*	NULL	NULL	NULL	NULL	NULL	NULL

Figure 7: Execution of Stored Procedure for adding income

3.5 User-Defined Functions

Custom SQL functions were engineered to compute real-time metrics, thereby reducing the computational load on the Python application. Notable functions include `fn_budget_status` (evaluating if a user is nearing their spending limit) and `fn_current_balance` (retrieving up-to-date account funds).

3.6 Automated Triggers

Triggers act as the automated core of the system's accounting logic:

- **Initialization:** Records an opening balance upon the creation of a new bank account.
- **Auto-Calculation:** Dynamically increments or decrements the `Balance` in `BankAccounts` following an `Income` or `Expense` insertion.
- **Auditing:** Automatically inserts a detailed log into the `BalanceHistory` table reflecting the exact state of the account before and after the transaction.

3.7 Backup & Recovery Protocols

To safeguard data, a complete backup script utilizing the `mysqldump` utility was implemented. The inclusion of `--routines` and `--triggers` flags ensured that all advanced database objects were successfully archived in the backup file.

```
C:\Program Files\MySQL\MySQL Server 8.4\bin>mysqldump -u root -p --routines --triggers PersonalFinanceDB > C:\Users\Public\PersonalFinanceDB_backup_final.sql
Enter password: *****
C:\Program Files\MySQL\MySQL Server 8.4\bin>
```

Figure 8: Database backup created successfully using `mysqldump`

4 Python Application Development

4.1 Database Connection Configuration

The CLI application interfaces with MySQL via the `mysql-connector-python` library. The connection is securely managed in the `db.py` module. Instead of utilizing the default root account, the application connects via a dedicated user role (`finance_admin`), adhering to best practices in database security.

4.2 Main CLI Application

The user interaction layer (`main.py`) provides a robust menu-driven interface offering 18 distinct functionalities. Users can easily navigate through options to add accounts, log transactions, review history logs, check their budget status, and generate analytical charts.


```
===== PERSONAL FINANCE MANAGEMENT SYSTEM =====
1. Add Income
2. Add Expense
3. View Monthly Summary
4. View Category-wise Spending
5. Check Budget Status
6. View Users
7. View Categories
8. View Bank Accounts
9. View Income History
10. View Expense History
11. Report: Income vs Expense Chart
12. Report: Category Spending Pie Chart
13. Add User
14. Add Bank Account
15. View Accounts by User
16. View Balance History
17. View Current Balance
0. Exit
Enter your choice: _
```

Figure 9: Command-Line Interface (CLI) of the system

4.3 Validation and Data Integrity Mechanisms

Validation occurs at two layers. On the application side (Python), `try-except` blocks handle `ValueError` exceptions to prevent crashes from invalid data types, while logical checks verify the existence and ownership of `AccountID` before allowing a transaction. On the database side, strict constraints and triggers reject invalid operations (e.g., preventing an expense if the account lacks sufficient funds).

4.4 Financial Analytics and Visualization

Integrating the `matplotlib` library, the application transcends text-based data by rendering visual financial analytics:

- **Bar Chart (Option 11):** Contrasts Total Income against Total Expenses for a specified year, offering an immediate overview of financial growth.
- **Pie Chart (Option 12):** Breaks down monthly spending by category, visually highlighting areas where the user spends the most.

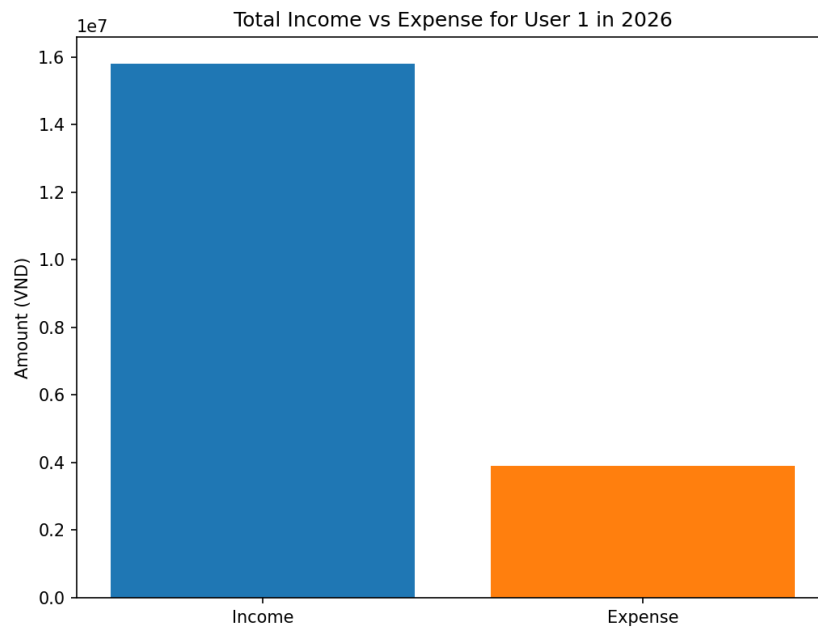


Figure 10: Income versus Expense Bar Chart generated by Python

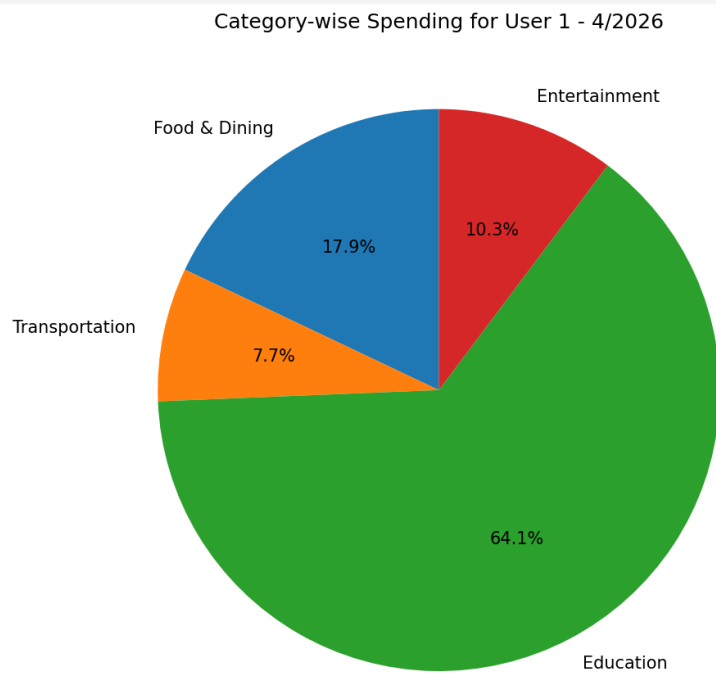


Figure 11: Category-wise Spending Pie Chart

5 Database Security and Administration

5.1 Comprehensive Audit Trails

The introduction of the `BalanceHistory` table is a critical security and administrative feature. It guarantees a transparent, tamper-proof historical record of where and when funds were moved, ensuring the system operates like a reliable banking ledger.

5.2 Access Control and Privileges

Application-level security is enforced by abandoning the `root` user for daily operations. A specialized account, `finance_admin` (identified by `Pfm@2026`), was established and granted specific privileges on the `PersonalFinanceDB` schema, minimizing security vulnerabilities.

5.3 Performance Optimization

As transaction volumes grow, table scans become computationally expensive. The strategic deployment of indexes on foreign keys and date columns significantly reduced the execution time for generating the `MonthlySummaryView` and rendering `matplotlib` charts.

6 Conclusion and Recommendations

6.1 Conclusion

The Personal Finance Management System successfully demonstrates the powerful integration of advanced relational database concepts with a Python-based application. By leveraging MySQL's complex features—such as Views, Stored Procedures, Functions, and Triggers—the system automates tedious accounting tasks, ensures strict data integrity, and maintains a flawless audit trail of balance histories. The Python CLI proves that the underlying database architecture is highly functional, successfully enabling users to log transactions, monitor budgets, and render visual financial reports efficiently.

6.2 Recommendations for Future Work

While the current command-line interface provides a fast and effective operational environment, future iterations of this project could include:

- Developing a Graphical User Interface (GUI) or a Web Application (using Flask or Django) for enhanced user experience.
- Implementing predictive analytics utilizing machine learning to forecast upcoming expenses based on historical data.
- Integrating features to export financial reports into PDF or Excel formats.

- Establishing API connections for real-time bank statement syncing.

7 References

- MySQL 8.4 Reference Manual.
- Python 3 Documentation.
- `mysql-connector-python` Official Documentation.
- matplotlib User Guide.
- Dinh Quoc Viet (2026). *Personal Finance Management System Source Code*. GitHub Repository: <https://github.com/vietdinh2209/personal-finance-management-system>